

TP Final

Aprendizaje por Refuerzo

Juan Pablo Alianak

GitHub: <https://github.com/jpalianak/rl1>

1. Introducción

El objetivo de este trabajo es aplicar el algoritmo de aprendizaje por refuerzo Q-Learning para resolver el entorno FrozenLake-v1, provisto por la librería Gymnasium. Este entorno simula un problema de decisión secuencial sobre un tablero de 4x4, donde un agente debe alcanzar una casilla objetivo evitando caer en agujeros. Las casillas del tablero pueden ser resbaladizas, lo que introduce estocasticidad en las acciones y dificulta el aprendizaje de una política óptima.

2. Descripción del entorno

El entorno FrozenLake-v1 provisto por Gymnasium representa un tablero de 4x4 con un total de 16 estados discretos. Cada estado corresponde a una casilla del tablero que puede tomar una de las siguientes formas:

- S: Inicio (*Start*)
- F: Piso congelado seguro (*Frozen*)
- H: Agujero (*Hole*, finaliza el episodio sin recompensa)
- G: Objetivo (*Goal*)

El agente puede moverse en cuatro direcciones: arriba, abajo, izquierda o derecha. Sin embargo, el entorno está configurado con el parámetro `is_slippery=True`, lo que introduce un comportamiento estocástico en los movimientos. Esto significa que la acción elegida por el agente no siempre se ejecuta con precisión, simulando un piso resbaladizo que puede desviarlo hacia una dirección distinta a la deseada. Esta característica añade un nivel adicional de complejidad al aprendizaje, ya que obliga al agente a desarrollar una política robusta frente a la incertidumbre de las transiciones.

3. Algoritmo de aprendizaje

Se implementó Q-Learning, un método de aprendizaje sin modelo (model-free) que actualiza una Q-Table donde cada celda representa el valor esperado de realizar una acción en un estado.

La fórmula de actualización de la Q-Table utilizada es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

donde:

- s es el estado actual
- a es la acción tomada
- r es la recompensa recibida
- s' es el nuevo estado
- a' son las posibles acciones desde s'
- α es la tasa de aprendizaje
- γ es el factor de descuento para recompensas futuras

4. Código

Durante el entrenamiento, en cada episodio el agente interactúa con el entorno y actualiza la Q-Table utilizando la siguiente estructura:

```
for episode in range(n_episodes):
    state, _ = env.reset()
    total_reward = 0
    steps = 0
    done = False
    for _ in range(max_steps):
        if np.random.uniform(0, 1) < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q[state, :])
        next_state, reward, done, truncated, info = env.step(action)

        # Actualizar Q-Table
        Q[state, action] += alpha * \
            (reward + gamma * np.max(Q[next_state, :]) - Q[state, action])
        state = next_state
        total_reward += reward
        steps += 1
        if done:
            break

    # Decaer epsilon
    epsilon = max(epsilon * epsilon_decay, epsilon_min)

    # Guardar métricas
    rewards.append(total_reward)
    steps_per_episode.append(steps)
    epsilons.append(epsilon)
```

Esta es la parte central del código donde el agente aprende de su experiencia, explorando el entorno y ajustando su política en función de las recompensas obtenidas. Si bien esta es la estructura base, en el repositorio es posible ver el código ampliado con todas sus variantes.

5. Resultados obtenidos

El entrenamiento se llevó a cabo en el entorno *FrozenLake-v1* con un tablero de 4x4 y piso resbaladizo. Inicialmente se utilizó el parámetro `is_slippery=False` para comenzar de menor a mayor en complejidad.

Los parámetros iniciales seleccionados fueron los siguientes:

- Tasa de aprendizaje (α) = 0.6
- Factor de descuento (γ) = 0.99
- Exploración inicial (ϵ) = 1.0
- Mínimo de ϵ = 0.01
- Decaimiento de ϵ = 0.9995
- Episodios = 20000
- Máximo de pasos = 100

La estrategia inicial de exploración utilizada fue ϵ -greedy, permitiendo al agente alternar entre la exploración de nuevas acciones y la explotación del conocimiento adquirido. A su vez, luego de entrenamiento, se realizó un test con 100 episodios para evaluar el resultado.

Esta configuración inicial permitió al agente rápidamente lograr un *reward* de aproximadamente 1.0 con una tasa de éxito de 100/100. En la figura 1 se observan los resultados iniciales.

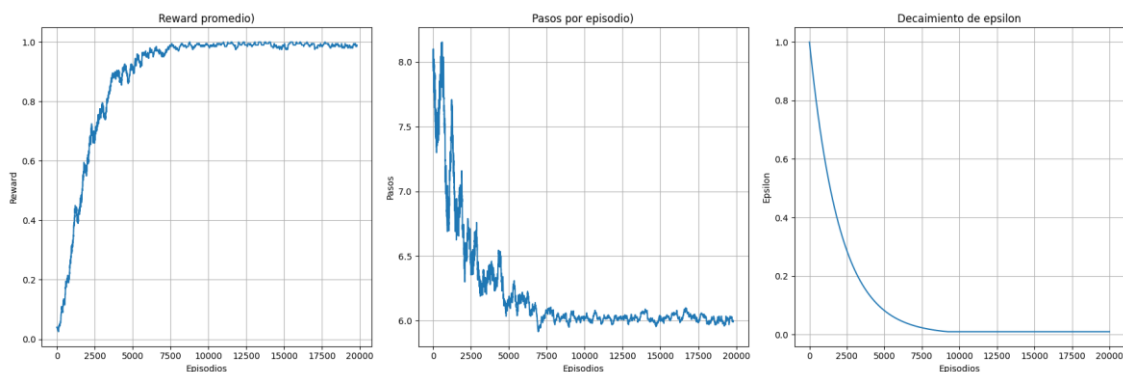


Figura 1: Reward, Pasos por Epsilon y Decaimiento de Epsilon

Luego de esta primera prueba inicial exitosa, se activó el parámetro `is_slippery=True` lo que introduce aleatoriedad en los movimientos del agente y hace más desafiante el entrenamiento.

Con la misma configuración de parámetros iniciales y con la inclusión de resbalamiento al agente logró un *reward* de aproximadamente 0.6 con una tasa de éxito de 59/100. En la figura 2 se observan los resultados iniciales.

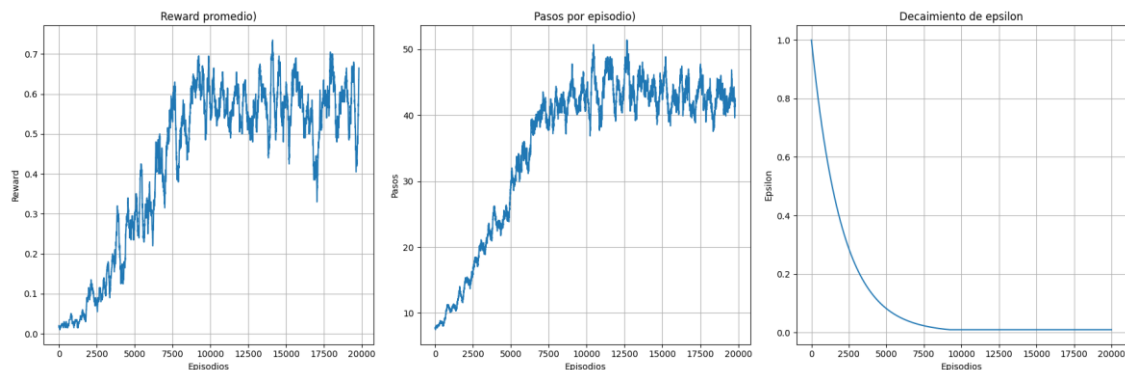


Figura 2: *Reward, Pasos por Epsilon y Decaimiento de Epsilon*

En la última figura se observa un estancamiento del *reward* alrededor de los 10000 episodios. Esto probablemente se deba al momento en donde el ϵ llega a su mínimo y el agente deja de explorar para dedicarse a explotar. Por este motivo la siguiente prueba consistió en cambiar el parámetro:

- Decaimiento de $\epsilon = 0.9999$
- Episodios = 50000

Con esta modificación los resultados fueron levemente mejores, logrando un *reward* de aproximadamente 0.7 con una tasa de éxito de 71/100. Estos resultados se observan en la figura 3.

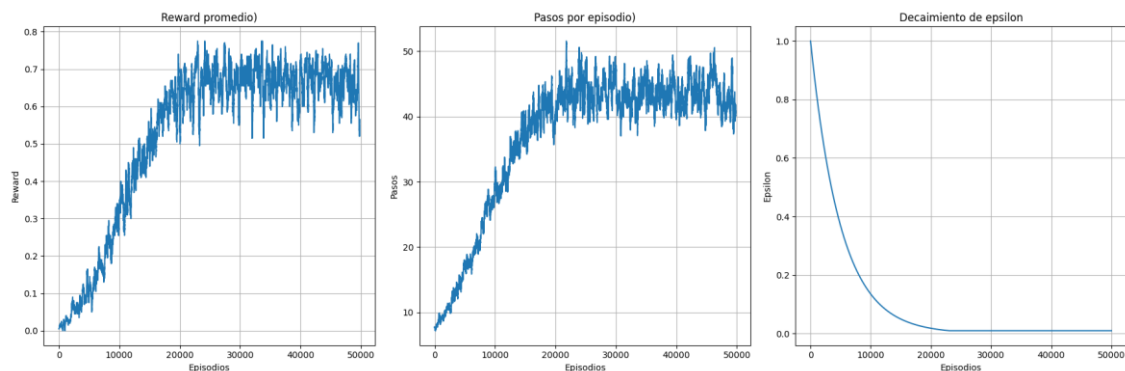


Figura 3: *Reward, Pasos por Epsilon y Decaimiento de Epsilon*

A su vez, se implementó dentro del algoritmo una estrategia de conservar la mejor Q-Table durante el entrenamiento para luego utilizarla en test.

Luego de estos resultados se probaron diferentes modificaciones de Alpha, Factor de descuento y Decaimiento. En todas estas pruebas no se observaron mejoras significativas. La mejor combinación de parámetros lograda fue la siguiente:

- Tasa de aprendizaje (α) = 0.3
- Factor de descuento (γ) = 0.995
- Exploración inicial (ϵ) = 1.0
- Mínimo de ϵ = 0.01
- Decaimiento de ϵ = 0.9998
- Episodios = 50000
- Máximo de pasos = 100

Luego de esta última configuración y dado el estancamiento que se observa en el *reward* se exploró la posibilidad de un reinicio parcial de ϵ si el *reward* no mejora luego de una cierta cantidad de episodios. Concretamente se implementó un reinicio del 10 % del ϵ máximo inicial en caso de que el *reward* de los últimos 500 episodios no mejora. Esto fue evaluado cada 20000 episodios. En la figura 4 se observan los resultados.

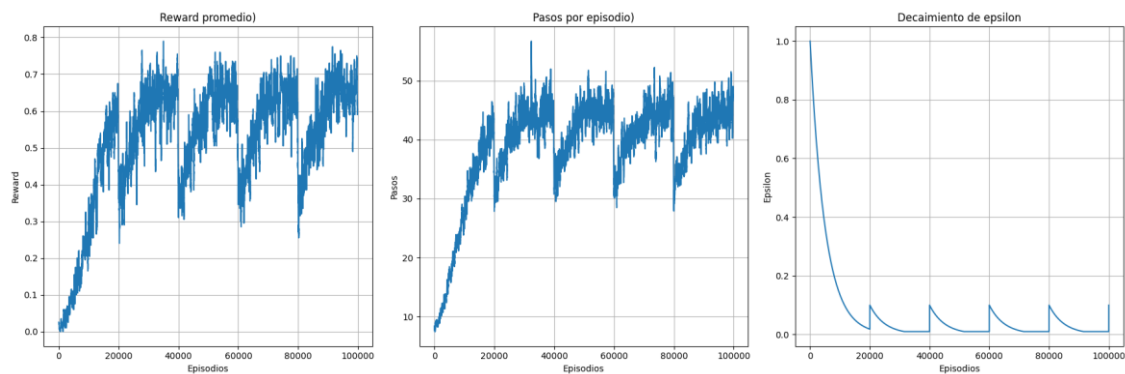


Figura 4: Reward, Pasos por Epsilon y Decaimiento de Epsilon

Se observa en la figura que aun incluyendo un reinicio para que el agente vuelva a explorar no se logra un nivel promedio de *reward* notablemente superior a 0.7.

Además de todas las pruebas anteriores, se implementó una versión del entrenamiento utilizando Optuna, una herramienta de optimización automática de hiperparámetros. El objetivo fue buscar combinaciones óptimas de tasa de aprendizaje, factor de descuento y parámetros de exploración de manera sistemática. Si bien la técnica permitió explorar múltiples configuraciones, no se

obtuvieron mejoras significativas respecto a los resultados alcanzados mediante el ajuste manual.

6. Conclusiones

- La aplicación de Q-Learning sobre el entorno FrozenLake-v1 permitió comprender el impacto que tienen la exploración, el aprendizaje y la estocasticidad del entorno en el proceso de entrenamiento. En condiciones deterministas (sin resbalamiento), el agente fue capaz de aprender rápidamente una política óptima, alcanzando una tasa de éxito del 100% con un *reward* promedio de 1.0.
- Sin embargo, al introducir aleatoriedad en las transiciones (*is_slippery=True*), la tarea se volvió considerablemente más desafiante. A pesar de implementar distintas estrategias de exploración, ajustes manuales de hiperparámetros, reinicios parciales del valor de ϵ y optimización automática con Optuna, no se logró superar consistentemente un *reward* promedio de 0.7.
- Se observó que el decaimiento de ϵ puede llevar al estancamiento del aprendizaje, dado que el agente deja de explorar caminos potencialmente mejores. Sin embargo, con las estrategias de reinicio parcial de ϵ para permitir retomar temporalmente la exploración no resultó en una mejora significativa del rendimiento final.
- Si bien Q-Learning es efectivo en entornos discretos y deterministas, su desempeño en contextos inciertos como FrozenLake con resbalamiento es limitado.
- Queda como trabajo futuro abordarlo con otras técnicas como SARSA, métodos con entornos modelo (model-based), redes neuronales o Deep Q-Learning.